



Tecnológico de Monterrey

Campus Querétaro

Alina Alejandra Reyes Muñiz-A01752345

Ian Alejandro Sierra Islas-A01706247

Daniel Guzmán Ochoa- A01770347

Eugenio Sánchez Velázquez- A00573129

Evidencia 6: Aplicación Web

Analítica de datos y herramientas de inteligencia artificial (Gpo 601)

12 de junio de 2026

Introducción

En este documento plasmamos las bases y todo el proceso que se realizó mediante las herramientas que nos aportaron gran valor en el desarrollo de nuestro proyecto.

Mencionamos las tecnologías usadas desde limpieza de datos, estandarización, librerías para analítica, modelos predictivos hasta la elaboración del dashboard.

Justificación de las tecnologías usadas:

- **Web/Dashboard:** Elegimos Dash porque nos permitió construir una aplicación web a la medida de la organización, uniendo las herramientas analítica y estadística de Python directamente con la interfaz del usuario todo en conjunto.

Al usar Dash, todo el ecosistema de análisis, desde la limpieza de datos en Pandas hasta las pruebas estadísticas avanzadas (como el ANOVA y los modelos de proyección) se conecta directamente al tablero web sin necesidad de cambiar de lenguaje, reescribir código o usar conectores externos de bases de datos.

Además, Dash permite programar lógica algorítmica compleja. Esto es lo que hace posible que, cuando un usuario interactúe con el mapa, el servidor ejecute en tiempo real modelos de demand sensing o segmentaciones personalizadas de forma inmediata.

Finalmente, Dash proporciona un entorno web basado en código abierto que permite personalizar cada tipografía y paleta de color institucional para cumplir con los estándares exactos de diseño ejecutivo (como la distribución limpia y las tarjetas de impacto financiero). Además, al ser una aplicación web autónoma, elimina la dependencia de licencias de visualización por usuario para la organización.

- **Modelos:**
 - `scipy.stats` y `statsmodels`: Se utilizaron para evaluar los supuestos del modelo (como la prueba de homogeneidad de varianzas de Levene), ejecutar el análisis de varianza robusto para varianzas heterogéneas (`sm_oneway`), realizar modelos de regresión lineal por fórmulas (`smf`) y validar proporciones (`proportions_ztest`).
 - `pingouin`: Se seleccionó específicamente por su capacidad de generar reportes detallados y realizar pruebas estadísticas robustas (como el ANOVA de Welch) con pocas líneas de código, entregando los tamaños del efecto de forma directa.
 - `pairwise_tukeyhsd`: Es la herramienta matemática responsable del análisis post-hoc. Una vez que el ANOVA demostró que existían diferencias globales,

Tukey permitió comparar los grupos par por par (Norte, Sur, Centro, Bajío, etapas del ciclo agrícola) para aislar cuáles regiones se comportan igual y cuáles son anomalías aisladas.

- `sklearn.preprocessing` (`StandardScaler`, `LabelEncoder`): Componentes críticos para la fase de ingeniería de características (`Feature Engineering`).
`LabelEncoder` transformó variables categóricas (como los nombres de los distribuidores o estatus) en formatos numéricos legibles para los modelos, mientras que `StandardScaler` estandarizó las variables numéricas (media = 0, varianza = 1), un paso obligatorio para algoritmos basados en distancias.
- `sklearn.cluster` (`KMeans`) y `silhouette_score`: Se utilizaron para la segmentación no supervisada de la flota. `KMeans` agrupó los tractores en función de sus perfiles de uso, mientras que el `Silhouette Score` aportó la métrica matemática objetiva para determinar la calidad de los clústers y validar el número óptimo de segmentos.
- `sklearn.linear_model` (`LogisticRegression`): Se implementó como el modelo base de clasificación para establecer una línea de referencia (baseline) estadística en la predicción de comportamientos (como la probabilidad de incumplimiento del servicio).
- `sklearn.model_selection` (`train_test_split`, `cross_val_score`): Garantizan la robustez metodológica de los modelos. `train_test_split` divide los datos para evitar el sobreajuste (`overfitting`), y la validación cruzada (`cross_val_score`) asegura que las métricas del modelo sean estables y generalizables a toda la flota nacional.
- `sklearn.metrics`: implementamos matrices de confusión, reportes de clasificación (precisión, recall, F1-score) y evaluación del error en regresión (`mean_absolute_error`, `mean_squared_error`, `r2_score`), lo que permite traducir la eficacia del algoritmo a métricas de certeza de negocio.
- `xgboost` (`XGBClassifier`, `XGBRegressor`): Representa el núcleo de analítica predictiva de alta capacidad de nuestro proyecto. Se eligió `Extreme Gradient Boosting` debido a que es el algoritmo más apto para datos tabulares y estructurados (como las bases de datos de telemetría y órdenes de servicio). Su arquitectura basada en árboles de decisión permite capturar relaciones no lineales complejas, manejar datos desbalanceados y mitigar el sesgo.

- Manipulación de Datos:

- pandas y numpy: Pandas se utilizó para la ingesta, limpieza, transformación y estructuración del archivo maestro de telemetría (en nuestro caso por decisión de eficiencia son dos archivos). Numpy aporta el soporte matemático para realizar operaciones de alta velocidad, manipulación de arreglos multidimensionales y gestión eficiente de valores nulos o faltantes.
 - os, datetime y timedelta: Se encargan de la interacción con el entorno del sistema (rutas de archivos y directorios) y del procesamiento avanzado de series de tiempo. Dado que la telemetría depende de cuándo ocurren las alertas, estas librerías permitieron calcular diferencias de tiempo, parsear fechas y estructurar las ventanas temporales de los ciclos agrícolas.
- **Visualización:**
- matplotlib, seaborn y gridspec: Nos aportaron la base y constituyeron el entorno de visualización técnica y exploración de datos del proyecto (EDA). Matplotlib fue de gran utilidad para tener las estructuras básicas pero esenciales, así como la posibilidad de manejar layouts más complejos (gridspec). Seaborn nos permitió tener mejor presentación de la estadística de alto nivel para graficar distribuciones complejas, mapas de calor de correlación y matrices de confusión con una estética limpia.
 - RocCurveDisplay: La utilizamos específicamente para evaluar el rendimiento de los clasificadores de Machine Learning, graficando el balance entre verdaderos positivos y falsos positivos (Curva ROC) para demostrar visualmente la certidumbre del modelo.

Instrucciones para correr la aplicación:

Como ya establecimos, nuestra aplicación está desarrollada con Dash, que nos permite unificar el análisis exploratorio de datos, el rendimiento de los modelos predictivos de Machine Learning y los indicadores clave de negocio (KPIs) en un entorno de producción local.

Para ejecutar la aplicación web hay que seguir los siguientes pasos:

1. Descargar la evidencia en su entorno local y correr los modelos predictivos, después colar el repositorio:

```
```bash git clone
```

```
https://github.com/xstorm28/CNH-Evidencia.git cd CNH-Evidencia
```

2. Preparación de las bases de datos: este paso es simplemente para garantizar que el servidor local no produzca un error de lectura, en caso de no encontrar el archivo. Al clonar el repositorio, el archivo maestro debería de encontrarse automáticamente dentro de la carpeta. Si hay modificaciones, simplemente hay que asegurarse de guardarlo en la raíz del proyecto con el nombre exacto de la base de datos.
3. Instalar las librerías mencionadas en el apartado anterior (estadísticas, manipulación de datos, y machine learning), asegurándonos de tener instalado 'pip' y todos los requerimientos: `pip install -r requirements.txt`
4. Para ejecutar el dashboard, verificamos que todas las librerías están instaladas y se inicial el servidor local del backend ejecutando el script principal de la aplicación: `python app.py`
5. Cuando la terminal indique que el servidor está corriendo activamente, en su navegador web ingrese a la siguiente dirección IP local:  
[\[http://127.0.0.1:8050/\]\(http://127.0.0.1:8050/\)](http://127.0.0.1:8050/)

### **Justificación de enfoque:**

Nuestro proyecto se enfoca en un modelo de demanda desagregada a nivel macro-regional y micro-estatal. Esta decisión está basada en una necesidad dictada por tres pilares de nuestro ecosistema de datos:

1. Calidad e Inconsistencia de los Datos (Dark Data)

Si se hubiera optado por un modelo de demanda total, el algoritmo habría sido severamente contaminado por el sesgo de la base de datos nacional. Al descubrir que el 79.3% de la flota opera bajo las sombras de la ceguera operativa (Dark Data), y que este problema no se distribuye de manera uniforme, un modelo global habría promediado realidades incompatibles.

La calidad de la información está altamente fragmentada: mientras que tres distribuidores específicos concentran el 45.4% de toda la ceguera del sistema, plazas clave a nivel estatal exhiben colapsos operativos locales en la captura de registros. Desagregar la demanda permite aislar los puntos de datos limpios de aquellos que requieren una intervención de limpieza previa.

## 2. Estructura y Varianza de los Datos de Telemetría

La infraestructura de los datos capturados por la telemetría y los horómetros demostró mediante pruebas estadísticas que la flota posee una estructura heterogénea y multifactorial que se rompe al intentar unificarla.

En el eje del tiempo (intensidad de uso), el ANOVA de Welch determinó que la macro-región Norte opera de manera aislada al resto del país, devorando el umbral preventivo en solo 4.5 meses frente a los casi 6 meses que le toma al Bajío. Un modelo de demanda total habría suavizado esta curva, enviando alertas tardías al Norte y prematuras al Centro-Bajío.

La prueba post-hoc de Tukey demostró que el Sur experimenta una velocidad de operación significativamente mayor al resto de las regiones. Esto altera el perfil estructural de desgaste (estrés dinámico por tracción en campo abierto frente a las operaciones de baja velocidad o estáticas del Centro y Bajío).

Modelar la demanda de Aftermarket de forma agregada habría resultado en un diseño algorítmico defectuoso que ignora que el desgaste no está ajustado al calendario, sino a la fatiga real de las unidades en entornos geográficos específicos.

## 3. Rendimiento y Capacidad Predictiva del Modelo

En términos de analítica predictiva, el uso de algoritmos avanzados de gradiente descendiente como XGBoost alcanza su máximo rendimiento cuando se le permite ramificar las reglas de decisión sobre subpoblaciones específicas.

Un modelo de demanda total habría sufrido de un alto sesgo, siendo incapaz de mapear las interacciones cruzadas entre variables categóricas (como la familia de equipos HERENCIA o la eficiencia de distribución local).

Al desagregar el enfoque, el algoritmo incrementa drásticamente su sensibilidad para predecir ventanas de oportunidad comerciales. Esto es lo que permite reducir el desfase de uso alto por debajo de las 300 horas y escalar la captura de servicios en ventana de menos del 5% actual hasta una meta del 30% a los 9 meses.

Enfocarnos en una demanda desagregada es lo que transforma este proyecto de un reporte descriptivo sin efecto a una herramienta de segmentación precisa. Capturar entre un 18% y 25% más ingresos en Aftermarket solo es posible porque el modelo no

asume un promedio nacional, lee con precisión matemática la velocidad de cada región, el rezago real de cada estado y el trabajo de cada unidad.

**Referencias:**

*Librerías en Data Science | Interactive Chaos.* (n.d.).

<https://interactivechaos.com/es/manual/tutorial-de-python/librerias-en-data-science>

All, M. (2025, December 23). *Las 31 mejores bibliotecas de Python para la ciencia de datos en 2026.* Datacamp.

<https://www.datacamp.com/es/blog/top-python-libraries-for-data-science>